

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Lecturer: Prof. Alessandro Ricci

[module-2.1]

EMBEDDED SYSTEMS AND OBJECT-ORIENTED MODELLING

SUMMARY

- This module considers aspects concerning the modelling of embedded software, using modelling paradigm such as OO

DESIGN AND DEVELOPMENT OF EMBEDDED SOFTWARE

- Two main modelling approaches
 - **top-down approach**
 - starting from the *domain* and its modelling using high-level paradigms, such as OO
 - **bottom-up approach**
 - starting from the hardware characteristics and related concepts
- This module is about a *top-down approach*

MODELLING AND PROGRAMMING

- Programming paradigms such as OO are first of all *modelling paradigms*
 - they specify how we can conceptualise, represent, model a system
 - howe we can analyse a problem and define the corresponding solution in some application domain
- **Model**
 - representation of relevant aspects of a system, abstracting from those that are considered not relevant
 - relevant aspects concern the structure, behaviour and interaction of the system
- Strong relationships between modelling and programming [OLE]
 - scandinavian school, where part of the OO is rooted

Ole Lehrmann Madsen, Birger Møller-Pedersen. *A Unified Approach to Modeling and Programming*, Model Driven Engineering Languages and Systems. Lecture Notes in Computer Science Volume 6394, 2010, pp 1-15.

IMPORTANCE OF MODELLING

- Analysing the problem and defining solutions in a more abstract way than implementation technologies and languages
- Advantages
 - better understanding of system working and management of related complexities
 - reusability, portability
 - extensibility
 - to have more rigorous approaches to verify the correctness of the system

MODELLING WHAT

- FUNDAMENTAL DIMENSIONS

- **Structure**
 - what are the parts composing a system
- **Behaviour**
 - the computational behaviour of every part
- **Interaction**
 - how parts interact

PARADIGMS AND LANGUAGES

- ***Modelling paradigms***
 - set of concepts and principles defining models
 - e.g.: **object-oriented paradigm**
 - often related to *programming*
 - es: OOP
- ***Modelling languages***
 - provide a rigorous non ambiguous way to represent models
 - e.g.: **UML language**
 - often related to specific modelling paradigms
 - e.g. UML has been strongly inspired by the OO paradigm, in particular for what concerns modelling the structure and interactions

OBJECT-ORIENTED PARADIGM

- Main paradigm in modelling and programming software
 - effective level of abstraction to capture and represent essential aspects of the problem and its domain
- Fostering important properties at the design and programming level
 - **modularity**
 - **encapsulation**
 - **reuse** and **extensibility** mechanism
- Among important aspects not directly captured by the paradigm
 - concurrency, asynchronous interactions, distribution

MODELLING EMBEDDED SOFTWARE BASED ON MICRO-CONTROLLERS

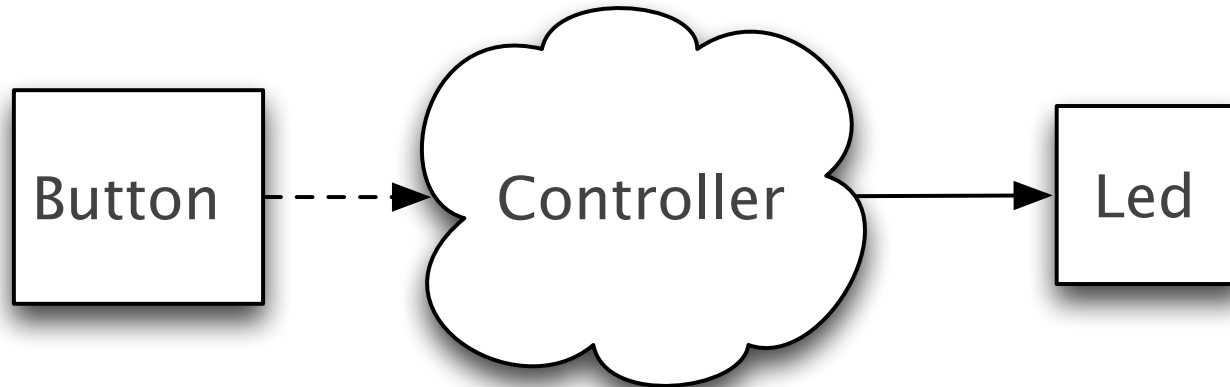
- Two main macro parts
 - **controller**
 - encapsulating the control/application logic concerning the task(s) to do
 - to this purpose, it accesses/uses/observes passive resources and devices
 - active component, based on some *control architecture*
 - **controlled elements**
 - modelling the resources and devices managed/used by the controller to do its job
 - e.g. I/O devices, sensors, actuators
 - encapsulates functionalities useful for the controller
 - passive component

FEATURES

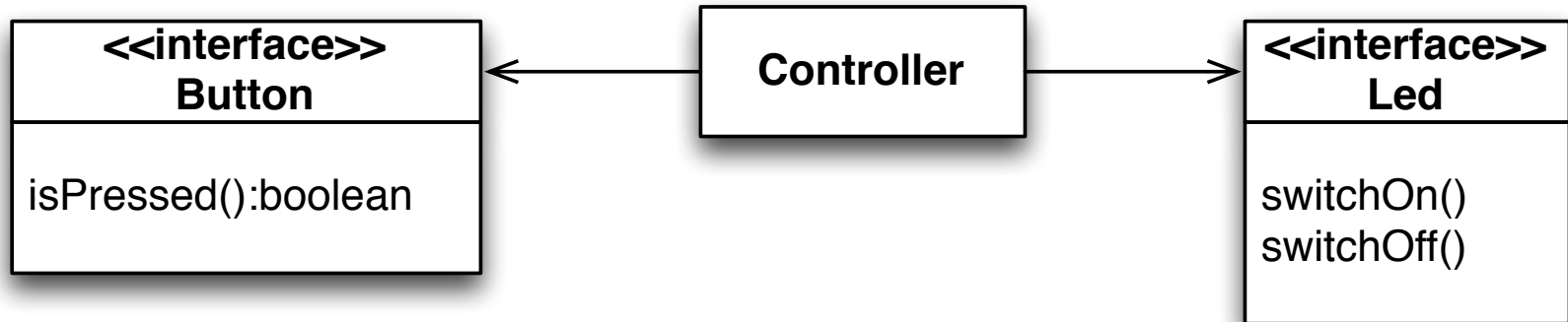
- **Controller**
 - Integrating pro-active and re-active behaviour
 - based on a *control loop* architecture
 - simplest one: super-loop
 - in a future module: event-loop
- **Controlled elements**
 - well-defined interface
 - observable state and events
 - reusability, controllability
 - OO model

BUTTON-LED EXAMPLE

- Modelling the *button-led* system
 - a button functioning as switch to control a led
 - the (software) controller must switch on the led when the button is pressed and switch off it when the button is released



AN OO MODEL OF THE BUTTON-LED



- Interfaces
 - Button
 - `isPressed(): boolean`
 - Led
 - `switchOn()`
 - `switchOff()`

CONTROLLER: CONTROL LOOP

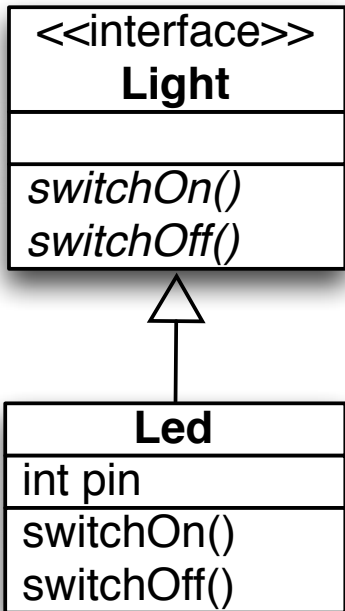
- **Control** loop — based on a super-loop architecture in this case

```
lightOn := false

loop {
    /* first: sensing */
    bool isPressed = button.isPressed();

    /* processing and acting */
    if (!lightOn && isPressed){
        light.switchOn()
        lightOn := true
    } else if (lightOn && !isPressed){
        light.switchOff()
        lightOn := false
    }
}
```

FROM MODELS TO CODE: ARDUINO IN WIRING/C++



```

#ifndef __LIGHT__
#define __LIGHT__

class Light {
public:
    virtual void switchOn() = 0;
    virtual void switchOff() = 0;
};

#endif
  
```

```

#ifndef __LED__
#define __LED__

#include "Light.h"

class Led: public Light {
public:
    Led(int pin);
    void switchOn();
    void switchOff();
protected:
    int pin;
};

#endif
  
```

```

/* Led.cpp */
#include "Led.h"
#include "Arduino.h"

Led::Led(int pin){
    this->pin = pin;
    pinMode(pin,OUTPUT);
}

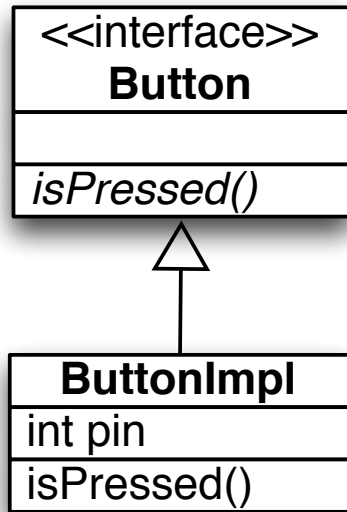
void Led::switchOn(){
    digitalWrite(pin,HIGH);
}

void Led::switchOff(){
    digitalWrite(pin,LOW);
};
  
```

FROM MODELS TO CODE: ARDUINO IN WIRING/C++

- Light interface (in Light.h)
 - interface for any lighting device — in C++: an abstract class
- Led device
 - concrete type of Light
 - declaration of the Led class in Led.h
 - definition/implementation of the Led class in Led.cpp
- Remarks
 - C++ allows for separating class declaration from class definition
 - to use/access Arduino base library/procedures/constants in *.cpp files, the header “Arduino.h” must be included
 - not needed in the *.ino main file

FROM MODELS TO CODE: ARDUINO IN WIRING/C++



```
#ifndef __BUTTON__
#define __BUTTON__
```

```
class Button {
public:
    virtual bool isPressed() = 0;
};

#endif
```

```
#ifndef __BUTTONIMPL__
#define __BUTTONIMPL__
```

```
#include "Button.h"

class ButtonImpl: public Button {
public:
    ButtonImpl(int pin);
    bool isPressed();

protected:
    int pin;
};

#endif
```

```
#include "ButtonImpl.h"
#include "Arduino.h"
```

```
ButtonImpl::ButtonImpl(int pin){
    this->pin = pin;
    pinMode(pin, INPUT);
}

bool ButtonImpl::isPressed(){
    return digitalRead(pin) == HIGH;
}
```


FROM MODELS TO CODE: ARDUINO IN WIRING/C++

- `Button`
 - basic button interface
- `ButtonImpl`
 - a class implementing buttons

FROM MODELS TO CODE: ARDUINO IN WIRING/C++

- Setup
 - objects creation
- Main (super-)loop

```
#include "Led.h"
#include "ButtonImpl.h"

#define LED_PIN    13
#define BUTTON_PIN 2

Light* light;
Button* button;
boolean lightOn;

void setup(){
    light = new Led(LED_PIN);
    button = new ButtonImpl(BUTTON_PIN);
    lightOn = false;
}

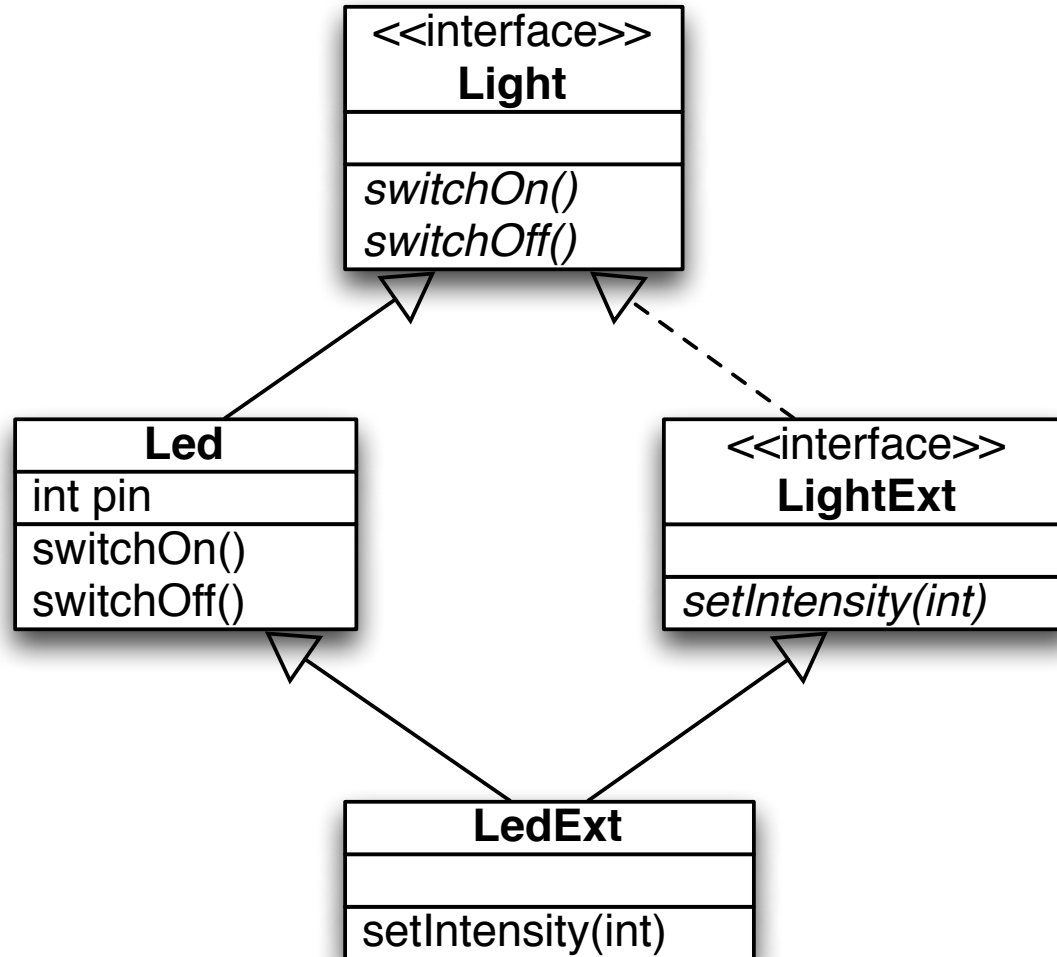
void loop(){
    bool isPressed = button->isPressed();
    if (!lightOn && isPressed){
        light->switchOn();
        lightOn = true;
    } else if (lightOn && !isPressed){
        light->switchOff();
        lightOn = false;
    }
}
```

REMARKS

- Object-Oriented level of abstraction
 - low-level implementation details (e.g. the specific wiring primitives used to drive pins) are hidden, not exposed to the controller
 - better readability, portability, reusability

EXTENSION: AN EXAMPLE

- Modelling a led with light intensity



EXTENSION: AN EXAMPLE

- Modelling a led with light intensity

```
#ifndef __LIGHT_EXT__
#define __LIGHT_EXT__
#include "Light.h"

class LightExt : public Light {
public:
    virtual void setIntensity(int) = 0;
};
#endif
```

```
#ifndef __LED_EXT__
#define __LED_EXT__
#include "Led.h"
#include "LightExt.h"

class LedExt: public LightExt, public Led {
public:
    LedExt(int pin);
    LedExt(int pin, int intensity);
    void switchOn();
    void switchOff();
    void setIntensity(int v);
private:
    int currentIntensity;
    bool isOn;
};
#endif
```

```
#include "LedExt.h"
#include "Arduino.h"

LedExt::LedExt(int pin) : Led(pin) {
    currentIntensity = 128;
    isOn = false;
}

LedExt::LedExt(int pin, int intensity) : Led(pin) {
    isOn = false;
    currentIntensity = intensity;
}

void LedExt::switchOn(){
    analogWrite(pin,currentIntensity);
    isOn = true;
}

void LedExt::setIntensity(int value){
    currentIntensity = value;
    if (isOn){
        analogWrite(pin,currentIntensity);
    }
}

void LedExt::switchOff(){
    analogWrite(pin,0);
    isOn = false;
}
```

FADING

```
#include "LedExt.h"

// the pin must be a PWM one
#define LED_PIN 9

LightExt* led;
int brightness;
int fadeAmount;

void setup(){
    brightness = 0;
    fadeAmount = 5;
    led = new LedExt(LED_PIN,brightness);
    led->switchOn();
}

void loop(){
    led->setIntensity(brightness);
    brightness = brightness + fadeAmount;

    if (brightness == 0 || brightness == 255) {
        fadeAmount = -fadeAmount ;
    }

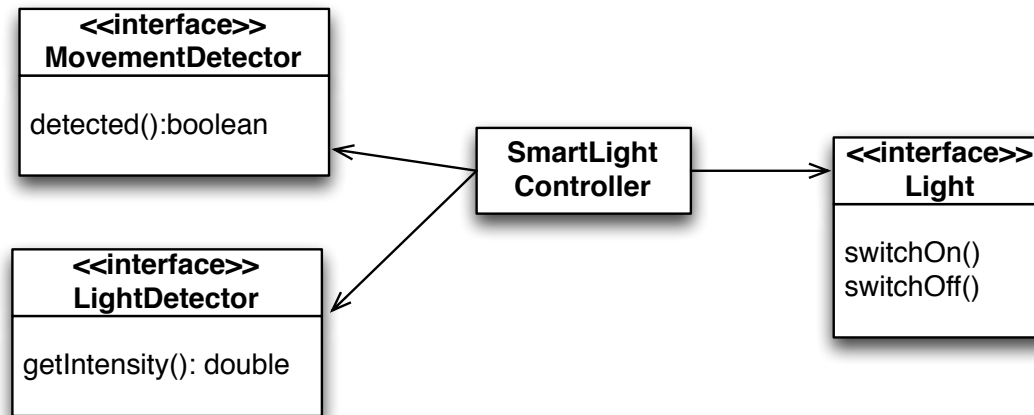
    delay(20);
}
```

REMARKS

- The new LedExt class can be reused also in previous examples, without changing the code

FROM A BUTTON-LED TO A SMART LIGHT

- As a simple variant, let's consider an environment equipped with a light that can be switched on or off depending on the presence of people around. Moreover, the light should be switched on even if the environment is dark
- Evolution of previous system
 - Light (output)
 - MovementDetector (input)
 - LightDetector (input)
 - SmartLightController



SMART LIGHT CONTROLLER

- Control loop

```
lightOn := false

loop {
    detected := presDetector.detected()
    intens := lightSensor.getIntensity()
    isLowIntens := intens < LIGHT_THRESHOLD
    if (!lightOn && detected && isLowIntens){
        light.switchOn()
        lightOn := true
    } else if (lightOn && (!isLowIntens || !detected)){
        light.switchOff()
        lightOn := false
    }
}
```

CONTROL LOOP AND FINITE STATE MACHINES

- Execution model based on a control loop (or super loop)
 - at each cycle the controller read the input state and choose the proper action to perform
 - the choice of the action depends on the current *state*
 - the action can lead to update the current state
- Need of having proper formalisms to describe state-based behaviours
 - **Finite State Machines (FSM)**
 - synchronous, asynchronous
 - next module

CONTROL LOOP: ABOUT EFFICIENCY AND REACTIVITY

- **Efficiency**
 - in the examples the loop is always cycling even if the controller is conceptually waiting for an input
- **Reactivity**
 - depends on how fast a cycle is completed
 - if during a single cycle sensors change states multiple times, these changes are not detected

ABOUT REACTIVITY: EXAMPLE

- *“Implement a controller for making a continuous fade in and fade out of a led, with duration DT seconds, using a button to start and stop asynchronously the process”*
- Simple solution ...

```
running := false
loop {
  if (button.switchedOn() &&
      !running){
    running := true
    light.switchOn()
  } else if (button.switchedOff()
              && running){
    running := false
    light.switchOff()
  }
  if (running){
    fadeIn()
    fadeOut()
  }
}
```

```
fadeIn(){
  intensity := 0
  delta := 1/NUM_STEPS;
  for (i := 0; i < NUM_STEPS; i++){
    intensity := intensity + delta
    light.setIntensity(intensity)
    sleep(DeltaTime)
  }
}

fadeOut(){...}
```

PROBLEM:

if the button changes during the fade in/out,
the controller does not detect it...

THE CONTROLLER AS AN ACTIVE ENTITY

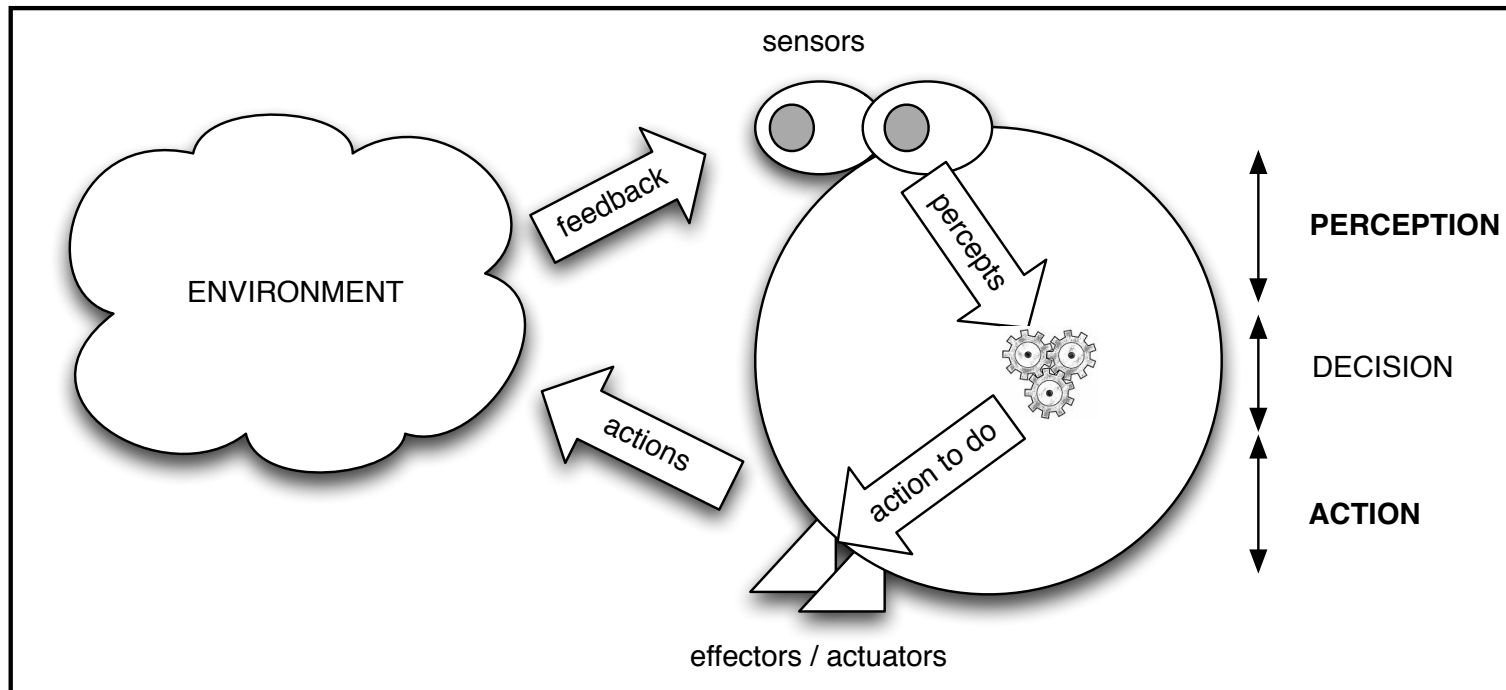
- About the conceptual nature of the controller
 - can it be modelled conceptually as an object in OO modelling?
- Actually objects in OO modelling/design/programming are typically passive entities
 - featuring proper interfaces to be used by the client of the object
 - they don't encapsulate a control flow
- The controller conceptually is an active entity, with its own logical control flow
 - at the design level, it is meant to encapsulate the control strategy of the system
- *So what kind of abstraction can be used to model this kind of entities?*

CONTROLLER MODEL: *AGENTS*

- We will use the concept of “**agent**” to model controllers
 - agents are first-class active entities, equipped with an autonomous logical control flow, designed to achieve or perform some task(s) that call for being reactive to data and events perceived from the environment (through sensors), and that do actions to have an effect on the environment through some actuators
- Agent as first-class design abstraction
 - reactive + pro-active behaviour
 - encapsulating a control flow
 - *task-oriented* design

AGENTS IN LITERATURE

- Concepts adopted in various contexts
 - Artificial Intelligence (AI) and Distributed AI
 - modelling and simulation
 - software engineering (Agent-Oriented Software Engineering)
 - Agents and Multi-Agent Systems Community [WOO,JEN]



CONTROL LOOP - TASK-BASED DECOMPOSITION AND EXECUTION

- Complex jobs can be decomposed into *tasks*, that are executed by the control loop
 - the agent carries on one or multiple tasks concurrently
 - the behaviour of each task can be described by a state machine
- This architecture is described in next modules

FROM INDIVIDUAL AGENTS TO MULTI-AGENT SYSTEMS

- Complex embedded systems can be distributed and actually include a system of interacting/cooperating independent sub-systems
 - e.g. a smart home, a smart city
 - Internet of Things (IoT) perspective
- In order to model these systems we can consider **multi-agent systems**, as ensemble of agents that interact and cooperate [WOO,JEN]

BIBLIOGRAPHY

- **[OLE]** Ole Lehrmann Madsen, Birger Møller-Pedersen. A Unified Approach to Modeling and Programming, Model Driven Engineering Languages and Systems. Lecture Notes in Computer Science Volume 6394, 2010, pp 1-15.
- **[GOF]** Erich Gamma; Richard Helm, Ralph Johnson, John M. Vlissides (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley
- **[WOO]** Michael Wooldridge. An Introduction to MultiAgent Systems. John Wiley & Sons
- **[JEN]** Nicholas R. Jennings. 2001. An agent-based approach for building complex software systems. Commun. ACM 44, 4 (April 2001), 35-41.